



MERRIMACK COLLEGE

CSC 6033

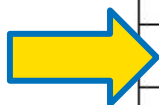
Week 6

Turing Machines

Languages, Automata, and Decidability - Dr. Paulo Fernandes

Timeline

Where are we?



Unit	Week	Topic	Projects	Quizzes	In-class Exercises	Exam
1	1	From Python to C++	P#1	Q#1	E#1	
1	2	Developing with C++	P#2	Q#2	E#2	
2	3	Finite Automata	P#3	Q#3	E#3	
2	4	Regular Expressions	P#4	Q#4	E#4	
3	5	Pushdown Automata	P#5	Q#5	E#5	
4	6	Turing Machines	P#6	Q#6	E#6	
5	7	Processing Input and Output	P#7	Q#7	E#7	
6	8	Decidability				Final Exam (all units)



MERRIMACK COLLEGE

Don't let tasks pile up!

Presentation Agenda

Week 6

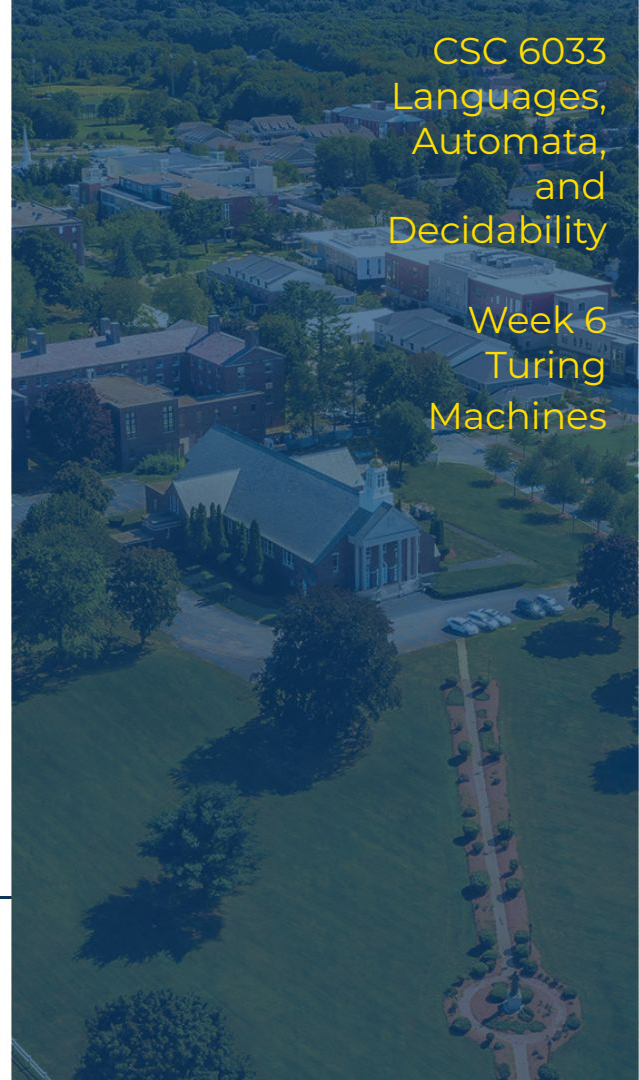
- Languages
 - a. Chomsky Hierarchy
 - b. Recursively Enumerable Languages
- Turing Machines
 - a. The Machine
 - b. Examples
- This Week's tasks



MERRIMACK COLLEGE

CSC 6033
Languages,
Automata,
and
Decidability

Week 6
Turing
Machines



Languages

“Human language appears to be a unique phenomenon, without significant analogue in the animal world.”

Noam Chomsky

We have seen regular languages and context-free languages, but there are some languages that can be more general than those.

Noam Chomsky and Marcel-Paul Schützenberger defined four levels of languages in what was later known as the:

- **Chomsky Hierarchy**

From which the higher level is:

- Recursively enumerable languages



Chomsky Hierarchy



Regular languages - **F**inite **S**tate **A**utomata

- a^*b^*

Context-free languages - **P**ush**d**own **A**utomata

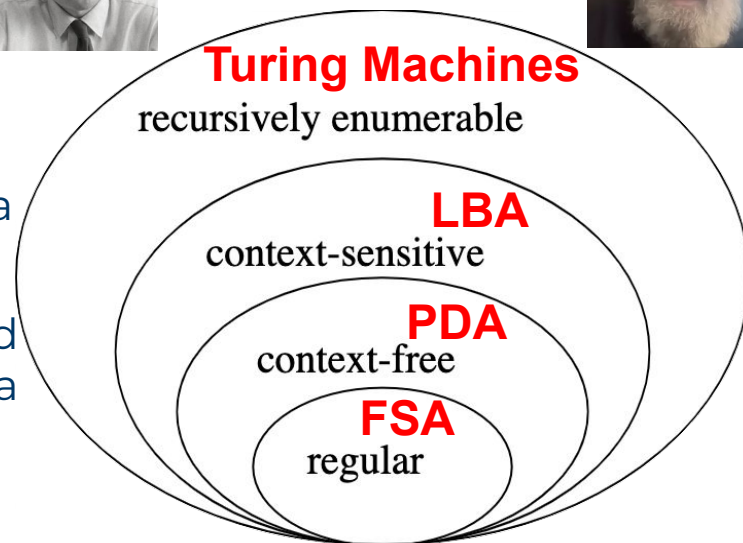
- $a^n b^n$

Context-sensitive languages - **L**inear **B**ounded **A**utomata

- $a^n b^n c^n$ (counting a 's and b 's with a stack, but how can we track the number of c 's?)

Recursively enumerable languages - **T**uring **M**achines

- $(a|b|c)^*$ with $\#a = \#b$ and $< \#c$ (it has to count a 's, b 's and c 's all the way until the end of the word, and only then check it out these numbers).



Chomsky Hierarchy

Regular languages - type-3 grammar

- a^*b^*

Context-free languages - type-2 language

- $a^n b^n$

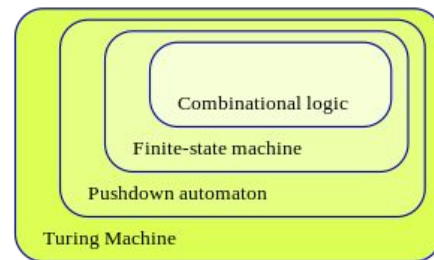
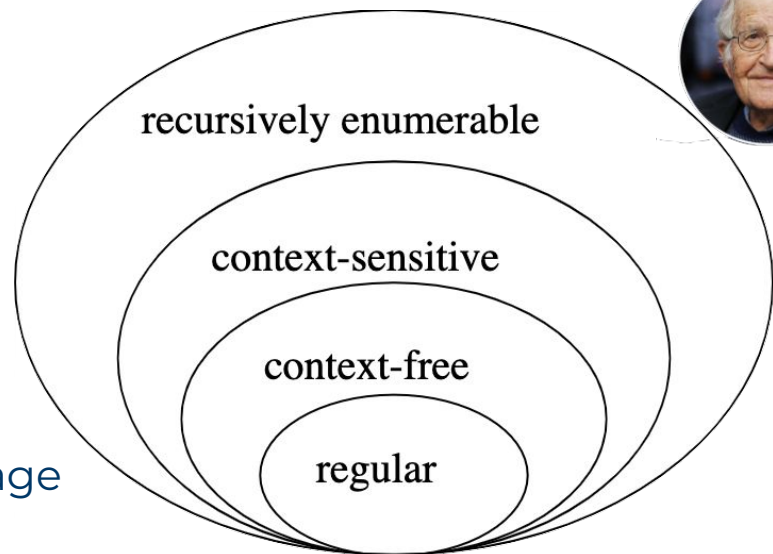
Context-sensitive languages - type-1 language

- $a^n b^n c^n$

Recursively enumerable languages - type-0 language

- $(a/b/c)^*$ with $\#a = \#b$ and $< \#c$

Curiously, Turing Machines were the first machines developed.



MERRIMACK COLLEGE

Video: [Chomsky hierarchy](#).

Languages

“Human language appears to be a unique phenomenon, without significant analogue in the animal world.”

Noam Chomsky

We have seen regular languages and context-free languages, but there are some languages that can be more general than those.

Noam Chomsky and Marcel-Paul Schützenberger defined four levels of languages in what was later known as the:

- Chomsky Hierarchy

From which the higher level is:

- **Recursively enumerable languages**



Recursively Enumerable (RE) Languages

- $(a/b)^*$ with $\#a = \#b$ as it has to keep track of the difference between the number of **a**'s and **b**'s.
- How would you decide if a given word belongs to that language?

```
19 int main() {
20     std::string input_str;
21
22     std::cout << "Enter a word: ";
23     getline(std::cin, input_str);
24
25     if (has_balanced_ab_greater_c(input_str)) {
26         std::cout << "The word belongs to the language." << std::endl;
27     } else {
28         std::cout << "The word does not belong to the language." << std::endl;
29     }
30     return 0;
31 }
```

```
1  #include <iostream>
2  #include <string>
3
4  int has_balanced_ab(std::string str) {
5      int count_a = 0;
6      int count_b = 0;
7      for (char ch : str) {
8          if (ch == 'a') {
9              count_a++;
10         } else if (ch == 'b') {
11             count_b++;
12         } else {
13             return false;
14         }
15     }
16     return (count_a == count_b);
17 }
```



Recursively Enumerable (RE) Languages

- $(a/b/c)^*$ with $\#a = \#b$ and $< \#c$
 - it has to count **a**'s, **b**'s and **c**'s all the way until the end of the word, and only then check it out these numbers.
- How would you decide if a given word belongs to that language?

```
22 int main() {
23     std::string input_str;
24
25     std::cout << "Enter a word: ";
26     getline(std::cin, input_str);
27
28     if (has_balanced_ab_greater_c(input_str)) {
29         std::cout << "The word belongs to the language." << std::endl;
30     } else {
31         std::cout << "The word does not belong to the language." << std::endl;
32     }
33     return 0;
34 }
```

```
1  #include <iostream>
2  #include <string>
3
4  int has_balanced_ab_greater_c(std::string str) {
5      int count_a = 0;
6      int count_b = 0;
7      int count_c = 0;
8      for (char ch : str) {
9          if (ch == 'a') {
10             count_a++;
11          } else if (ch == 'b') {
12             count_b++;
13          } else if (ch == 'c') {
14             count_c++;
15          } else {
16             return false;
17          }
18      }
19      return (count_a == count_b) && (count_a < count_c);
20 }
```



Turning Machines

“A computer would deserve to be called intelligent if it could deceive a human into believing that it was human.”

Alan Turing

Alan Turing defined a theoretical machine, called **a-machine**, as for automatic machine.

His PhD student Alonzo Church referring to this a-machine, use the name Turing Machine (TM), to honor his former advisor.

The name was largely adopted, and it was formally proven that if you can do it in a computer language, you can do it in a TM, and vice-versa.

- **The Machine**
- Examples



Turing Machines - TM

A TM is a theoretical model of computation that manipulates symbols on a tape according to a set of rules.

A TM consist of:

- A tape divided into cells, one next to the other;
- A head that can read and write symbols on the tape and move the tape left and right one cell at a time;
- A state transition table, given the current state of the machine and the symbol under the head, may tell the machine to:
 - Either erase or write a symbol in the tape cell under the head;
 - Move the head one cell left or one cell right;
 - Move from the current state to another one.



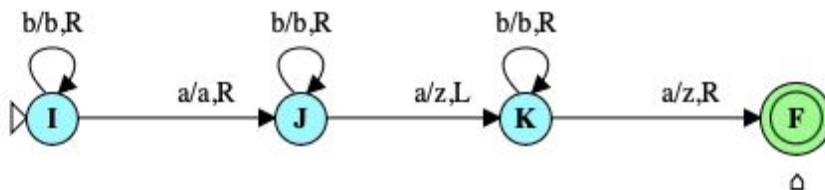
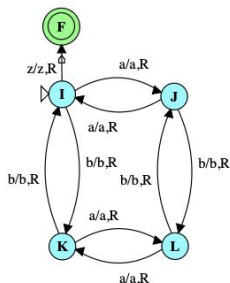
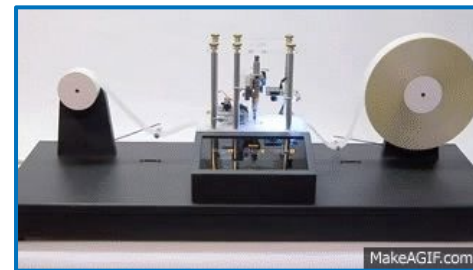
MERRIMACK COLLEGE

Video: [Turing Machines - How Computer Science Was Created By Accident.](#)

Turing Machines - TM

Practically, a TM is defined by a state transition table as a graph with states as vertices and edges with annotation stating:

- The symbol in the **tape** cell under the **head**;
- The symbol to be written at the **tape** cell under the **head**;
- The **tape** movement towards the left (**L**) or the right (**R**).



a/a, R

Video: [Turing Machines](#).

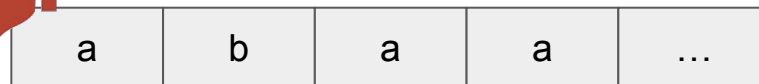
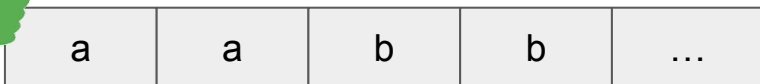
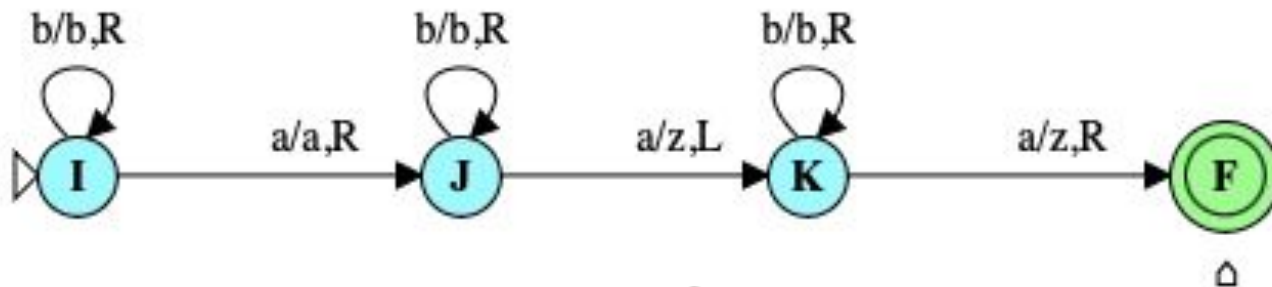
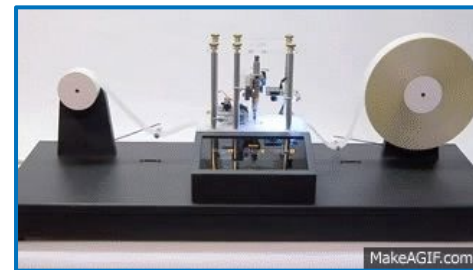
Text: [Turing Machine](#).



MERRIMACK COLLEGE

Turing Machines - TM

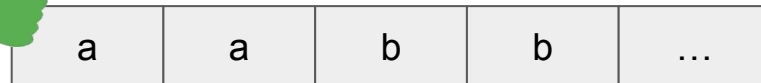
For example, the TM below accept words where the first letter **a** has to be followed by another letter **a**.



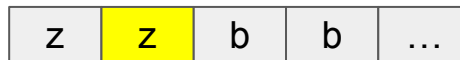
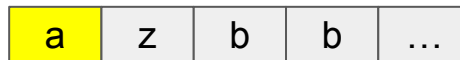
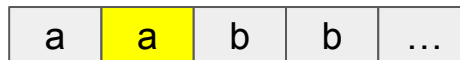
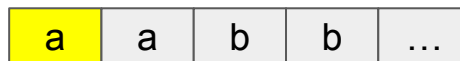
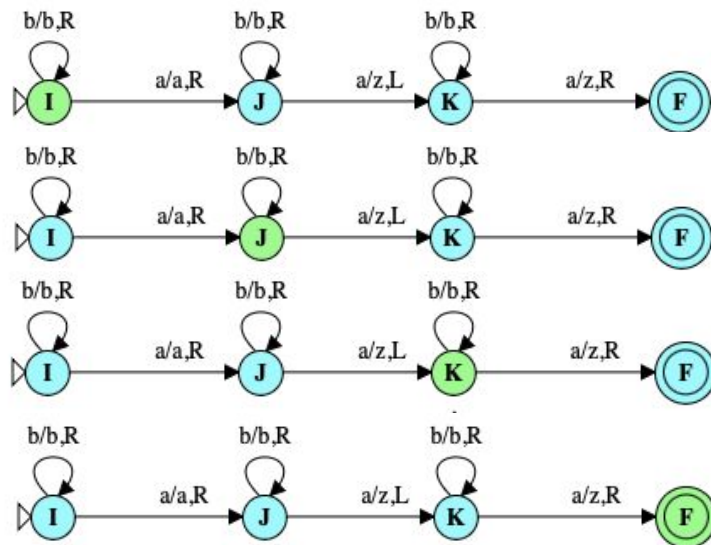
MERRIMACK COLLEGE

Tool: [Formal Language Editor - Old Dominion Univ.](#)

Turing Machines - TM



TM accepts words where the first letter **a** has to be followed by another letter **a**.

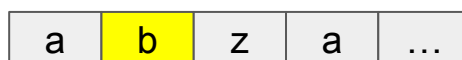
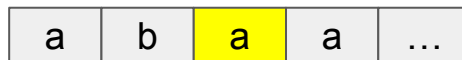
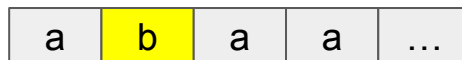
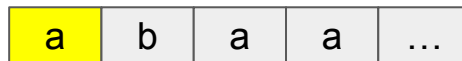
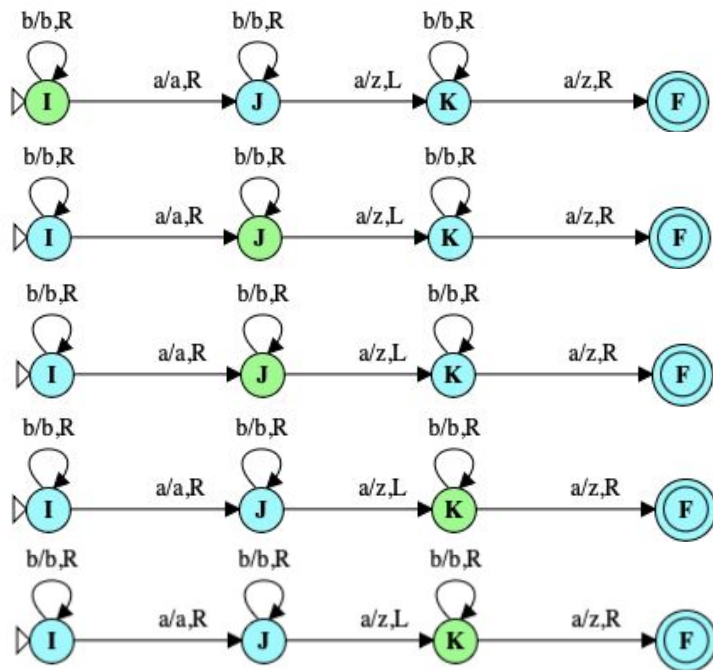
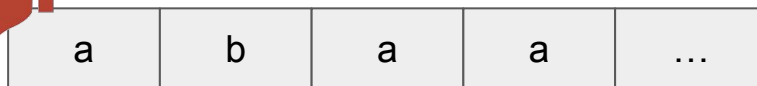


The head position at the end is not important.

Reaching the acceptance final state suffices.



Turing Machines - TM



The head position at the end is not important.

Not reaching the acceptance final state suffices.



MERRIMACK COLLEGE

Execution towards rejection.

Turning Machines

“A computer would deserve to be called intelligent if it could deceive a human into believing that it was human.”

Alan Turing

Alan Turing defined a theoretical machine, called **a-machine**, as for automatic machine.

His PhD student Alonzo Church referring to this a-machine, use the name Turing Machine (TM), to honor his former advisor.

The name was largely adopted, and it was formally proven that if you can do it in a computer language, you can do it in a TM, and vice-versa.

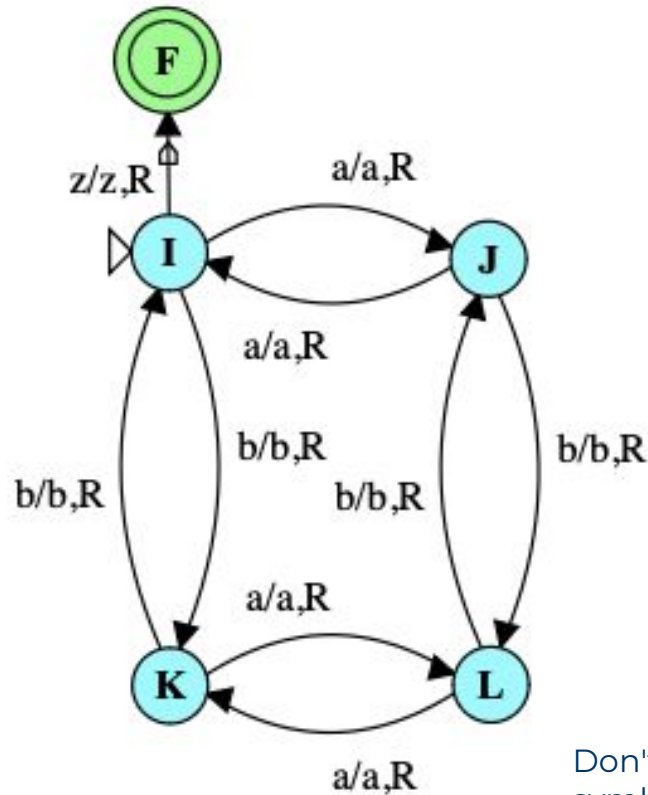
- The Machine
- **Examples**



TM - Example 1

The TM here accepts words with even number of **a**'s and **b**'s, ending with the symbol **z**.

- State **I** has an even number of **a**'s and **b**'s;
- State **J** has an odd number of **a**'s and even number of **b**'s;
- State **K** has an even number of **a**'s and an odd number of **b**'s;
- State **L** has an odd number of **a**'s and **b**'s.



Use the Formal Language Editor to create this Turing Machine and execute it step by step for the words:

- **abba**
- **bbaa**
- **aba**
- ϵ
- **aa**
- **bba**
- **bbb**
- **abbbba**

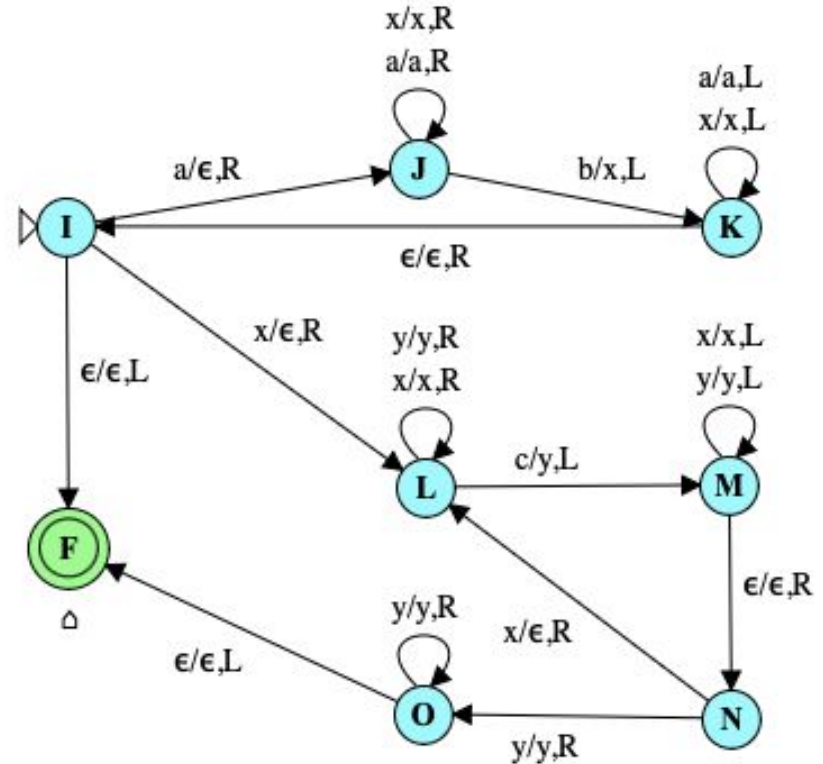
Don't forget the **z** symbol at the end.



TM - Example 2

The TM here accepts words with the general format $\mathbf{a^n b^n c^n}$.

- States **I**, **J**, and **K** accept equal number of **a**'s and **b**'s, replacing by this number of symbols **x**;
- States **L**, **M**, and **N** accept equal number of **x**'s and **c**'s, replacing it by this number of symbols **y**;
- State **O** eliminates the symbols **y**, leading to the acceptance.



This Week's tasks

- In-class Exercise E#6
- Coding Project P#6
- Quiz Q#6

Tasks

- Create a Turing Machine that accepts an odd number of a 's and an even number of b 's.
- Create a Turing Machine that receives two unary numbers and add it.
- Quiz #6 about this week topics.



In-class Exercise - E#6

Considering the Example 1 (see slides 17) as your starting point, create a Turing Machine (TM) that accepts only words with an odd number of a's and an even number of b's.

- Feel free to use the Formal Language Editor Tool to create your TM;
- Test your TM with the words:
 - ***ababa***
 - ***abba***
 - ***bababa***

Your TM graph should be displayed in a single **.pdf** file together with the execution for the three example words execution.

You have to submit the **.pdf** file with your work on Canvas within the deadline.



MERRIMACK COLLEGE

This task counts towards the In-class Exercises grade
and the deadline is This Saturday.

Sixth Coding Project - P#6

Create a Turing Machine that receives two unary numbers and add it. Inspire yourself in the explanation available in these sources:

- <https://www.geeksforgeeks.org/turing-machine-addition/>
- <https://www.tutorialspoint.com/design-a-tm-to-compute-addition-of-two-unary-numbers>

Try your TM for the following examples: $2+3$, $5+1$, and $5+6$.

Your TM graph should be displayed in a single **.pdf** file together with the execution for the three example words execution.

You have to submit the **.pdf** file with your work on Canvas within the deadline.



MERRIMACK COLLEGE

This assignment counts towards the Projects grade and the deadline is Next Tuesday.

Sixth Quiz - Q#6

- The sixth quiz in this course covers the topics of Week 6;
- The quiz will be available this Saturday, and it is composed by 10 questions;
- The quiz should be taken on Canvas (Module 6), and it is not a timed quiz:
 - You can take as long as you want to answer it;
- The quiz is open book, open notes, and you can even use any language Interpreter to answer it;
- Yet, the quiz is evaluated and you are allowed to submit it only once.

Your task:

- Go to Canvas, answer the quiz and submit it within the deadline.



MERRIMACK COLLEGE

This quiz counts towards the Quizzes grade and the deadline is Next Tuesday.

” Week 6 of CSC 6033

- **Do In-class Exercise E#6 until Saturday;**
- **Do Quiz Q#6 (available Saturday) until next Tuesday;**
- **Do Coding Project P#6 until next Tuesday.**

Next Week - Processing Input and Output



MERRIMACK COLLEGE